
◆ AEM Content Sync

DEPLOYMENT & OPERATIONS RUNBOOK

Deployment & Publishing Runbook

Install, deploy, wire the permissions, promote to a Production workspace, and publish the App Builder app to your organization's catalog — for AEM as a Cloud Service.

PROJECT

AEM Content Sync

REPOSITORY

github.com/rwunsch/aem-content-sync

APP TYPE

App Builder · dx/excshell/1

LICENSE

Apache-2.0

1 · What this is

The Content Sync app wraps Cloud Manager's **Content Copy** in a scheduled, observable, role-gated pipeline that keeps a lower environment (Stage) aligned with Production — including the *published* state, which Content Copy alone does not carry across.

It is an Adobe **App Builder** application (a `dx/excshell/1` Experience Cloud Shell extension) with a React SPA front end and Adobe I/O Runtime actions behind it. It runs entirely on Adobe infrastructure; there are no customer-hosted servers.

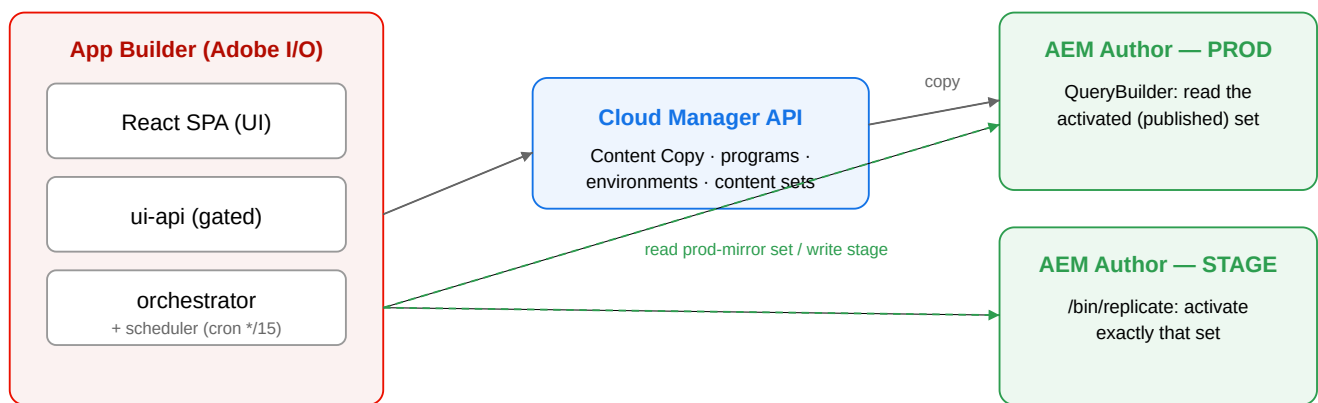


Figure 1 — The app holds both environments' credentials and talks to Cloud Manager (copy) and each AEM author (prod-mirror publish) directly.

Why "prod-mirror" publishing

Cloud Manager Content Copy is **author-tier only** and **strips replication status** (`cq: lastReplicated*`, `cq: lastReplicationAction*`, `cq: lastReplicatedBy*`) — by design, because that metadata describes the *source's* publish tier. So after a copy the destination author has no record of what was published, and nothing is published on Stage. The app's **prod-mirror** mode reads Production's activated set (QueryBuilder, per agent: publish/preview) and replicates exactly that set on Stage — reproducing production's live published state every run.

2 · The permission model (read this first)

Three independent authorization concerns, each granted in a different place. Getting one wrong fails in a different way — so understand them before you start.

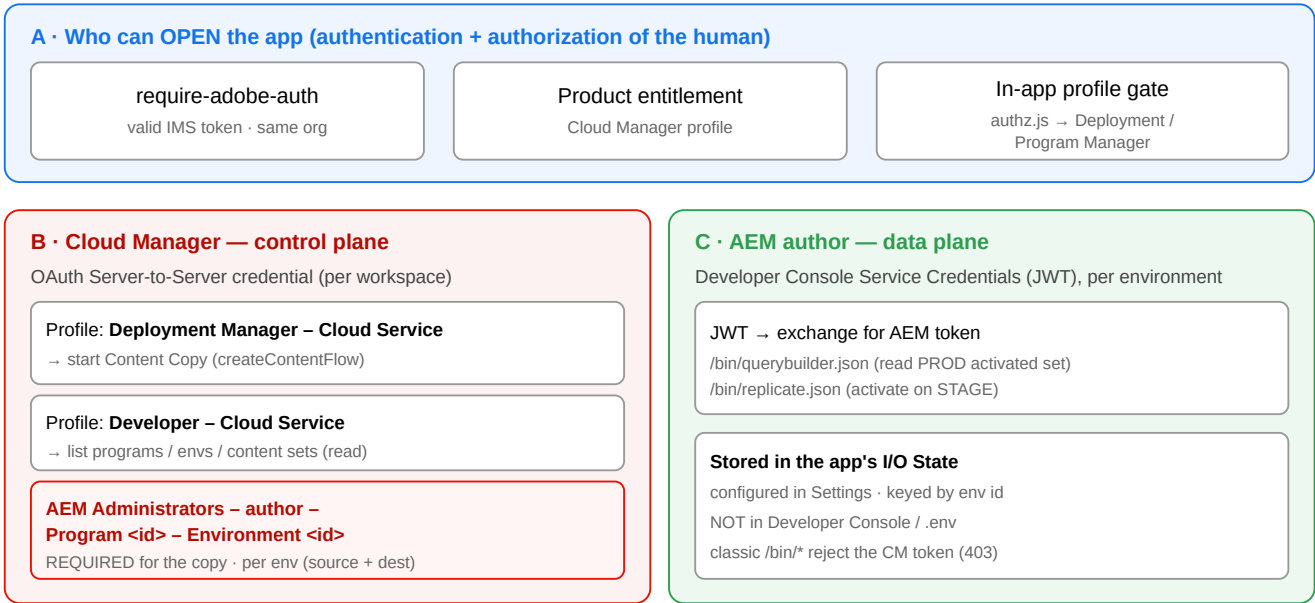


Figure 2 — How the permissions fit together. **B** drives Cloud Manager (the copy); **C** drives the AEM authors (the prod-mirror publish); **A** controls who may open the app at all.

The one that bites everyone

The Cloud Manager *Deployment Manager* profile is **necessary but not sufficient** for Content Copy. The API credential must **also** be a member of the `AEM Administrators – author – Program <id> – Environment <id>` product profile, for **both** the source and destination environments. Read calls succeed without it; only `createContentFlow` checks it — so the classic symptom is *"listing works but the copy returns 403."*

The two-token model

Token	How it's obtained	Used for
Cloud Manager token	<code>client_credentials</code> grant at <code>ims/token/v3</code> , using the S2S client id + secret	Cloud Manager API — copy, programs, environments, content sets
AEM author token	signed JWT exchanged at <code>ims/exchange/jwt</code> , using the per-environment Service Credentials	Classic AEM <code>/bin/replicate</code> & <code>/bin/querybuilder</code> (prod-mirror publish)

3 · Step-by-step — Stage

1 Developer Console project & workspace

Create (or open) the App Builder project. It has a **Stage** workspace by default; you'll add **Production** later (§5). Add the APIs the app needs: **Cloud Manager**, **I/O Management**, **State**.

2 Cloud Manager credential + profiles

On the workspace, add an **OAuth Server-to-Server** credential to the Cloud Manager API and attach the product profiles **Deployment Manager – Cloud Service** and **Developer – Cloud Service** (Figure 2-B).

3 AEM Administrators profile — per environment easy to miss

In the **Admin Console**, add this same API credential to the `AEM Administrators – author – Program <id> – Environment <id>` product profile for **both** the source and destination environments (Products → that profile → *API credentials* → *Add API credentials*). Allow ~10–30 min to propagate.

4 Pull config & deploy

```
# project-local aio (matching Parcel) – always use ./node_modules/.bin/aio
npm install
aio app use          # select the Stage workspace; writes .aio + .env
aio app deploy       # builds SPA + actions, deploys, publishes the extension
```

The deploy prints the in-shell URL. Open it via the Experience Cloud Shell — never the raw `adobeio-static.net` URL (the shell injects the IMS token the gated actions need).

5 Map the credential into action inputs

`aio app use` writes the IMS S2S *context* but **not** the convenience vars the actions read. Set them in `.env` from the context values (strip the brackets on the secret), then redeploy:

```
IMS_ORG_ID      = AIO_ims_contexts_<ctx>_ims__org__id
CM_CLIENT_ID    = AIO_ims_contexts_<ctx>_client__id
CM_CLIENT_SECRET = AIO_ims_contexts_<ctx>_client__secrets # strip [ ]
```

6 Configure jobs & AEM credentials in the app

Open the app → **Settings**: paste each environment's AEM **Service Credentials** JSON (Figure 2-C). → **Configure**: pick the program, source→destination environments, content set(s), publish mode (**prod-mirror**), and the cron schedule. → **Access control**: confirm the allowed Admin Console profiles (default: Deployment + Program Manager).

4 · Promote to Production & publish to the catalog

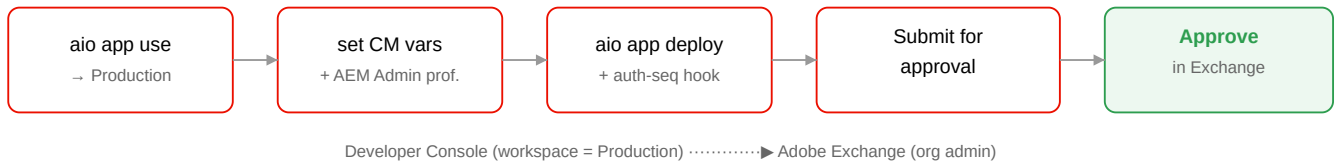


Figure 3 — Production promotion and catalog publish flow.

1 Switch the project to Production

```
aio app use -w Production --overwrite --no-input # rewrites .env/.aio to the Production namespace
```

Back up first

Switching **overwrites** the local `.env` / `.aio`. Copy them aside so you can return to Stage dev with `aio app use -w Stage`.

Production needs its **own** Cloud Manager S2S credential and — as in §3 steps 2–3 — the **Deployment Manager** profile *and* the per-environment **AEM Administrators** profiles. Re-map the CM convenience vars (§3 step 5) for the Production credential.

2 Deploy to Production

```
aio app deploy
```

Fresh-namespace gotcha (auto-healed)

On a brand-new namespace the deploy can create the inner `__secured_ui-api` action but skip the `require-adobe-auth ui-api wrapper sequence` → the SPA's calls 404 → UI shows "backend unreachable." The app ships a `post-app-deploy` hook (`scripts/ensure-auth-sequence.js`) that re-creates it automatically after every deploy.

3 Seed the Production app's configuration

The Production namespace starts empty. Configure jobs, AEM Service Credentials and access profiles in the app exactly as in §3 step 6 (Production has its own I/O State, independent of Stage).

4 Submit for publishing

Developer Console → **Production** workspace → **Approval** → fill the submission form (App title, description, contact email, 512×512 icon, note to reviewer) → **Submit**. Status becomes *In Review*.

5

Approve (org admin)

exchange.adobe.com/manage → **App Builder applications** → **Private** tab → find the app (*Pending Review*) → **Review** → **Approve**. Status flips to **Approved**.

6

Result — stable catalog URL

The app appears under the org's App Builder Apps with a durable, reloadable URL (no more `devMode`):

```
https://experience.adobe.com/#/@<org>/custom-apps/<namespace>/
```

End users still need the product entitlement *and* one of the configured Admin Console profiles (Figure 2-A).

5 · Run & monitor

A run is triggered by the scheduler (cron `* /15` evaluates each job's own schedule) or manually from the UI. Each run is a small state machine; the orchestrator runs one phase per invocation and self-chains, so no single action exceeds the 60-second I/O Runtime limit.

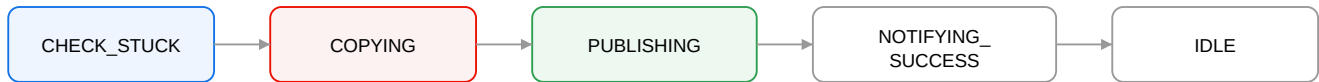


Figure 4 — Run state machine. COPYING calls Cloud Manager; PUBLISHING drives the prod-mirror replication; failures route to NOTIFYING_FAILURE → IDLE.

Progress, per-job status and the last ~50 log lines are visible in the UI and persisted in I/O State. Content-flow status from Cloud Manager is `IN_PROGRESS / COMPLETED / FAILED / CANCELLED`.

Where to look when something fails

403 on the copy → AEM Administrators per-env profile missing (§2). **"Backend unreachable"** → auth-wrapper sequence (§4 step 2). **401 everywhere** → opened outside the shell, no IMS token. **403 "not authorized: profile"** → caller lacks an allowed Admin Console profile.